

Simulación de Sistemas P con Paralelismo Basado en Prioridades en Mathematica

Izan Pinto Fortea

Simulación de Sistemas P con Paralelismo Basado en Prioridades en Mathematica

0. Planteamiento

En este trabajo se presenta el diseño e implementación de un simulador de Sistemas P desarrollado en *Wolfram Mathematica*. El modelo implementado permite representar estructuras jerárquicas de membranas, aplicar reglas de reescritura con prioridades y ejecutar el sistema bajo una estrategia de paralelismo máximo. La propuesta incorpora además mecanismos de disolución de membranas, comunicación arbitraria entre regiones y finalización por estado estable o por límite de iteraciones.

El simulador constituye una aproximación práctica al paradigma de computación por membranas, permitiendo experimentar con sistemas distribuidos inspirados biológicamente mediante una representación recursiva compacta y flexible.

1. Introducción

La computación por membranas, también conocida como computación con Sistemas P, es un modelo de computación inspirado en el funcionamiento de las células biológicas. Introducido por Gheorghe Păun a finales de los años noventa, este paradigma utiliza estructuras jerárquicas de membranas que contienen objetos y reglas de transformación.

Un sistema P puede entenderse como un conjunto de compartimentos organizados jerárquicamente, donde los objetos evolucionan de acuerdo con reglas locales y pueden desplazarse entre membranas. Una de las características fundamentales de estos sistemas es el paralelismo masivo: múltiples reglas pueden ejecutarse simultáneamente en diferentes regiones.

El objetivo de este trabajo es presentar una implementación funcional de un simulador de Sistemas P en Mathematica, capaz de:

- Representar sistemas de membranas arbitrariamente anidados.
- Gestionar reglas con prioridades.
- Ejecutar reglas de forma no determinista.
- Simular disolución de membranas.
- Permitir comunicación libre entre las membranas.
- Detener la simulación por convergencia a estado estable o límite de iteraciones.

2. Fundamentos Teóricos - Sistemas P

Formalmente, un Sistema P puede describirse mediante:

$$\Pi = (O, \mu, w_1, \dots, w_n, (R_1, \rho_1), \dots, (R_n, \rho_n), i_0)$$

Donde:

- O es el alfabeto de objetos
- μ es la estructura de membranas
- w_i es el multiconjunto inicial de objetos en la membrana i
- (R_i, ρ_i) es el conjunto de (reglas, prioridades) asociadas a la membrana i
- i_0 es la membrana de salida

En este trabajo se utilizará una variante simplificada donde:

- Los objetos se representan mediante caracteres minúsculos.
 - Las reglas poseen prioridades explícitas y no jerárquicas.
 - La comunicación entre membranas no requiere adyacencia.
 - Existe un símbolo δ para disolver membranas.
-

3. Representación del Sistema

3.1 Estructura General

El sistema completo se representa mediante la estructura:

```
Sistema_P =  
{  
    ESTADO_INICIAL,  
    REGLAS,  
    OUTPUT  
}
```

Donde:

- ESTADO_INICIAL representa la estructura de membranas con los símbolos ya embebidos.
- REGLAS contiene las reglas y prioridades asociadas a cada membrana.
- OUTPUT indica la membrana cuya información constituye la salida.

3.2 Representación de Membranas

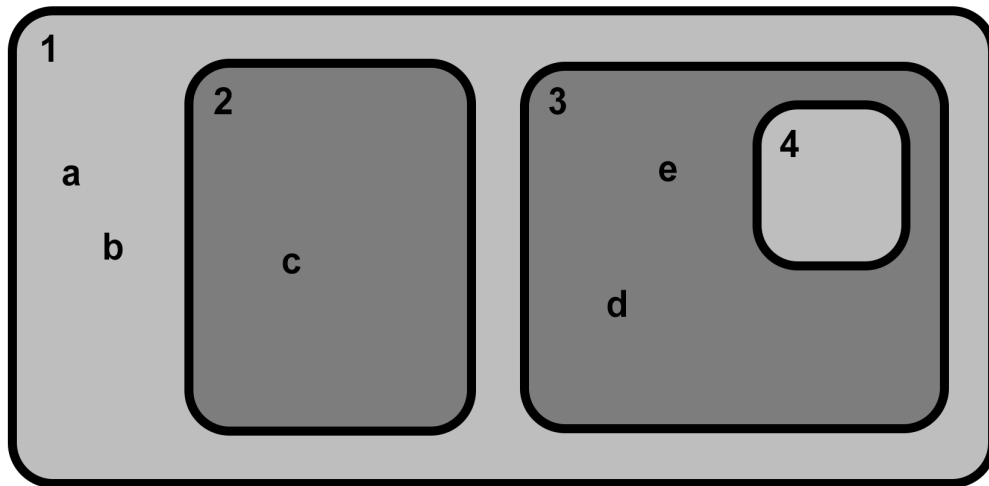
Cada membrana tiene que representar un multiconjunto. Aunque Mathematica no dispone de multiconjuntos nativos, éstos serán representados mediante listas con repetición de elementos. Cada lista se modelará mediante el identificador de la membrana en la primera posición, seguido por los objetos que contenga, seguido por las sub-membranas que contenga:

```
{ID_membrana, objeto_1, objeto_2, ..., submembrana_1, ...}
```

Por ejemplo:

```
sistema_de_membranas =  
{1, a, b, {2, c}, {3, d, e, {4}}}
```

representa la siguiente estructura de membranas:



La elección de listas recursivas facilita enormemente:

- la navegación jerárquica
- la búsqueda de membranas
- la disolución de membranas

3.3 Representación de Reglas

Cada regla tiene la siguiente forma:

```
{{antecedentes},{consecuente_1, where_1, ...}}
```

Por ejemplo:

```
regla_1 = {{a,b},{c,1,d,2}}
```

representaría la siguiente regla:

$$ab \rightarrow c_{in1} d_{in2}$$

Consumir en la membrana actual a y b , producir c en la membrana 1 y producir d en la membrana 2.

Si una regla produce uno o más símbolos δ , su correspondiente *where* puede tomar cualquier valor, ya que la membrana a disolver es la actual. Por convención y por el bien de la claridad, recomendamos igualmente poner el *ID_membrana* de la membrana a disolver en el *where*.

3.3 Prioridades y Reglas

Una vez ya tenemos un conjunto de reglas, tendremos que agruparlas dentro de una misma membrana, junto a sus prioridades:

```
{ID_membrana, {regla_1, ...}, {prioridad_1, ...}}
```

Las prioridades serán un número entero, dónde la *prioridad_i* corresponde a la *regla_i*, dicho *i* determinado por la posición (index) en las respectivas listas. De darse el caso en el que una membrana no tiene reglas, no será necesario hacer un conjunto vacío de reglas, se podrá simplemente ignorar dicha membrana (más adelante se pone un ejemplo).

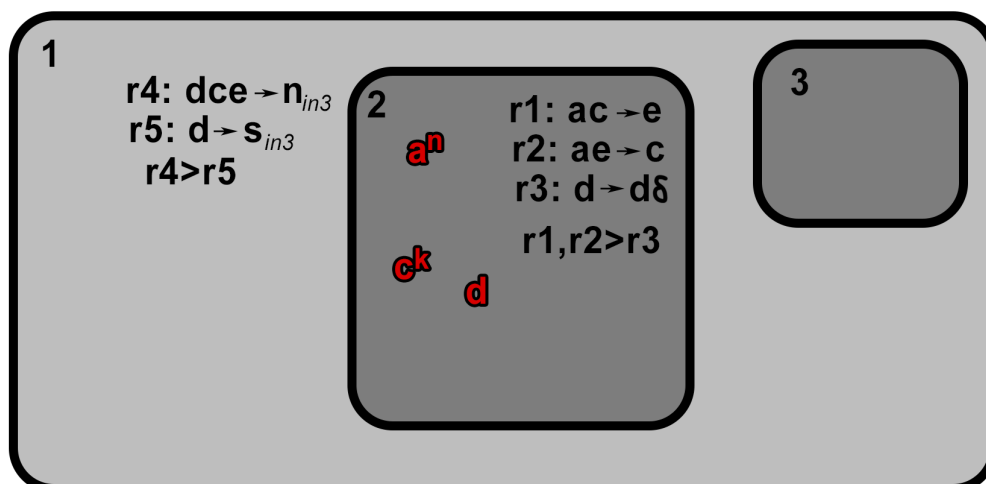
El algoritmo siempre intenta aplicar reglas con la mayor prioridad disponible. Si varias reglas comparten la misma prioridad y se pueden llevar a cabo, se selecciona una aleatoriamente, introduciendo un comportamiento no determinista.

Para una membrana, en una misma iteración de ejecución se llevarán a cabo la máxima cantidad de reglas posibles, siempre y cuando dichas reglas tengan la **misma prioridad**, de no tener la misma prioridad, las de menor prioridad, aun pudiendo efectuarse, serán ignoradas.

Este mecanismo aproxima el comportamiento típico de los Sistemas P máximamente paralelos.

3.4 Ejemplo

Vamos a definir ahora un sistema P que podría ser utilizado con el simulador para computar si $n \% k = 0$ es cierto o falso. Dicho sistema P puede ser representado gráficamente de la siguiente manera con la membrana final $i_0 = 3$:



Y en el modelo de Mathematica se representaría de la siguiente manera:

```

system= {1, {2,  $a^n$ ,  $c^k$ , d}, {3}};
rule1 = {{a, c}, {e, 2}};
rule2 = {{a, e}, {c, 2}};
rule3 = {{d}, {d, 2,  $\delta$ , 2}};
rule4 = {{d, c, e}, {n, 3}};
rule5 = {{d}, {s, 3}};
prioritiesMembrane2 = {2, 2, 1}; (*r1,r2>r3*)
prioritiesMembrane1 = {2, 1}; (*r4>r5*)
rules = {
{2, {rule1, rule2, rule3}, prioritiesMembrane2},
{1, {rule4, rule5}, prioritiesMembrane1}
};

```

O en forma compacta:

```

system= {1, {2,  $a^n$ ,  $c^k$ , d}, {3}};
rules = {{2, {{a, c}, {e, 2}}, {{a, e}, {c, 2}}, {{d}, {d, 2,  $\delta$ , 2}}}, {2, 2, 1}}, {1, {{d, c, e}, {n, 3}}, {{d}, {s, 3}}}, {2, 1}}};

```

Y posteriormente para ejecutar la simulación simplemente pasamos esos dos parámetros y la membrana final como tercer parámetro:

```

P = {system, rules, 3};

```

Luego en la salida se mostrará la traza indicando en cada iteración qué reglas han sido aplicadas para cada membrana, y una vez pare, qué objetos se encuentran en la membrana de output.

4. Arquitectura del simulador

4.1 Función Principal: *Simulate*

La función “main” del problema es:

```
Simulate[P_, iters_]
```

Su objetivo consiste en inicializar el sistema, ejecutar ciclos de evolución, detectar convergencia, detener la simulación y extraer el contenido de la salida. La simulación terminará cuando se alcance un estado estable, o se supere el número máximo de iteraciones (*iters*). Hasta que llegue el momento de parar, la función *Simulate* actualizará el estado del sistema llamando iterativamente a la función *ExecutionCycle* con el estado actual del sistema.

4.2 Ciclo de Ejecución

Cada vez que *ExecutionCycle* es llamado con un estado del sistema *P*, dado un estado:

$$C_t$$

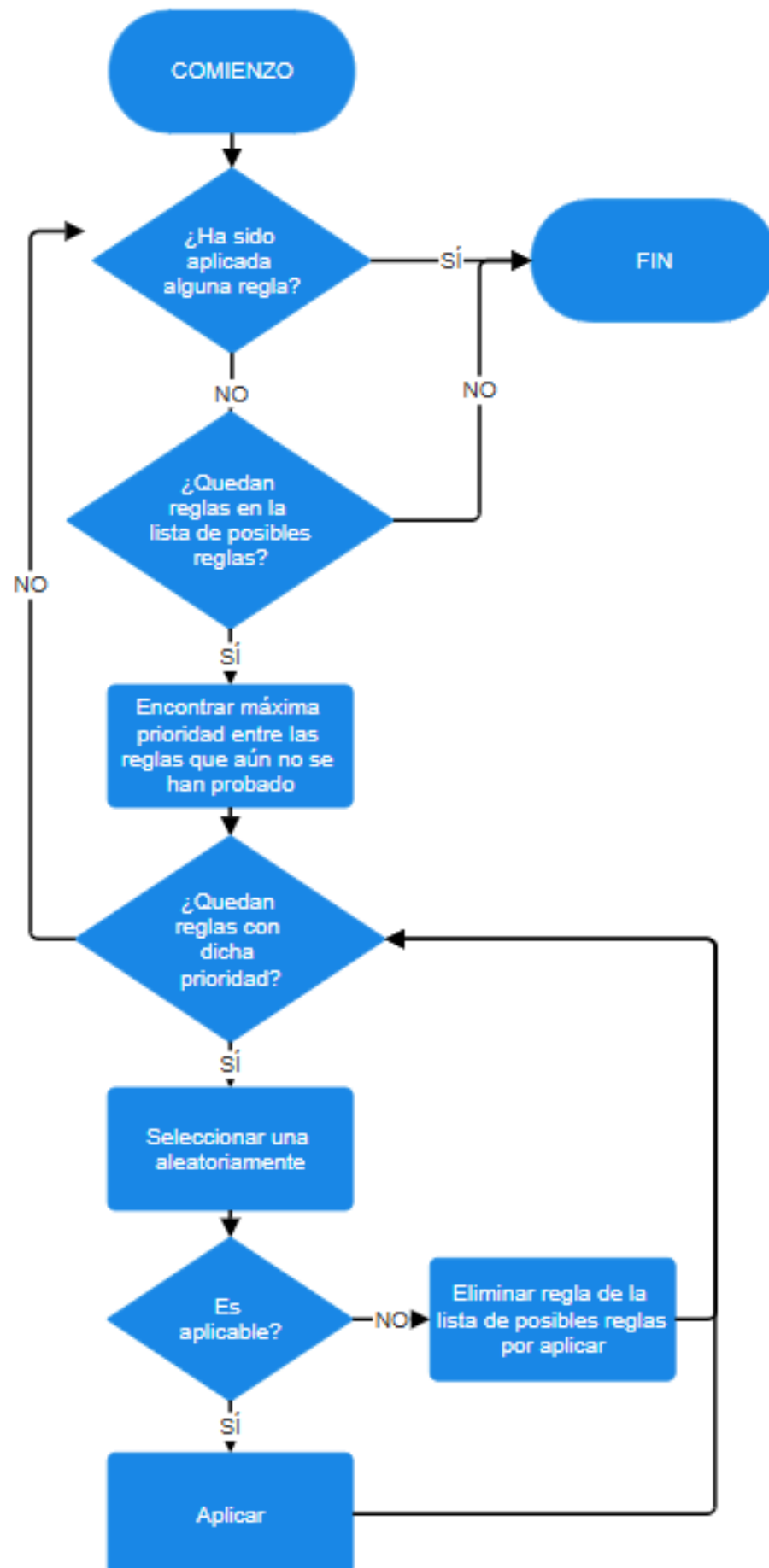
la aplicación de un conjunto de reglas compatibles produce un nuevo estado:

$$C_t \Rightarrow C_{t+1}$$

Para hacer esto, recorrerá todas las membranas del sistema de manera recursiva con *Depth-First Search*, y al llegar a una membrana que no contenga más sub-membranas que aún no hayan sido simuladas, llamará a la función *MembraneSimulation* con los parámetros oportunos para dicha membrana, y combinará las respuestas de la simulación de todas las membranas por separado en un único nuevo estado, el cual devolverá a *Simulate*.

4.3 Simulación de una Membrana

La simulación de una membrana es el núcleo del programa, y lo que hace es implementar el mecanismo de evolución. Este método consta de varias fases, así que es más claro si se representa con un diagrama de flujo. Para cada membrana dada, junto a su conjunto de reglas, la función *MembraneSimulation* sigue los pasos detallados en la siguiente página:



El funcionamiento interno de la función *MembraneSimulation* puede entenderse mediante el siguiente procedimiento iterativo :

1. Inicialmente se comprueba si ya ha sido aplicada alguna regla durante la simulación de la membrana actual. Si ya se aplicó al menos una regla (*y no quedan más reglas compatibles de la prioridad correspondiente*), la ejecución termina.
 2. Se mantiene una lista dinámica de reglas candidatas, inicialmente compuesta por todas las reglas asociadas a la membrana. Durante la ejecución, aquellas reglas que se demuestran inaplicables son eliminadas temporalmente de dicha lista.
 3. A partir de las reglas restantes, se construye una sublista *listaReglasMaxPrio* formada exclusivamente por las reglas con mayor prioridad disponible.
 4. Mientras existan reglas dentro de *listaReglasMaxPrio* , se selecciona una de manera aleatoria.
 5. Una vez seleccionada una regla, se comprueba si todos sus antecedentes están presentes en la membrana actual.
 6. Si la regla es aplicable:
 - se consumen sus antecedentes
 - se generan sus consecuentes en las membranas correspondientes
 - y, si aparece el símbolo especial δ , la membrana se marca para disolución y posteriormente se disuelve.
 7. Tras aplicar una regla, el algoritmo continúa intentando aplicar nuevas reglas de la misma prioridad máxima mientras sigan siendo compatibles con el estado actualizado de la membrana.
 8. Si una regla no puede aplicarse, se elimina de *listaReglasMaxPrio*, ya que con el estado actual del sistema no podrá ejecutarse durante esa iteración.
 9. El proceso finaliza cuando:
 - no quedan reglas aplicables con la prioridad máxima actual, o
 - ninguna regla puede ejecutarse en la membrana.
-

5. Conclusión / Resumen

En este trabajo se ha presentado una implementación funcional de un simulador de Sistemas P en Wolfram Mathematica, capaz de representar estructuras jerárquicas de membranas, aplicar reglas con prioridades y modelar mecanismos de evolución inspirados en la computación por membranas.

La arquitectura desarrollada utiliza una representación recursiva basada en listas anidadas, lo que permite gestionar de forma sencilla tanto la navegación entre membranas como operaciones dinámicas tales como la inserción y eliminación de símbolos o la disolución de regiones. Gracias a ello, el simulador puede representar sistemas arbitrariamente complejos manteniendo una estructura compacta y flexible.

El modelo también incorpora comportamiento no determinista mediante selección aleatoria entre reglas de igual prioridad, acercándose así al comportamiento habitual de muchos Sistemas P teóricos. Además, la posibilidad de comunicar objetos entre membranas arbitrarias simplifica el diseño de sistemas complejos y facilita la experimentación computacional.

La implementación realizada demuestra que Mathematica constituye una herramienta especialmente adecuada para este tipo de simulaciones, debido a su capacidad para manipular estructuras recursivas y expresiones simbólicas de manera natural. Asimismo, la modularidad del código facilita futuras extensiones del simulador.

Entre las posibles mejoras futuras destacan la optimización de las búsquedas recursivas mediante estructuras más eficientes, la incorporación de visualización gráfica de la evolución del sistema y la inclusión de nuevos mecanismos clásicos de computación por membranas, como división de membranas o reglas probabilísticas.

En conjunto, el simulador desarrollado constituye una base sólida para continuar investigando modelos bioinspirados de computación y explorar experimentalmente el comportamiento de Sistemas P dentro de un entorno de programación simbólica.
